DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING – DATA SCIENCE
ABES ENGINEERING COLLEGE, GHAZIABAD

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: FlightFix: Aircraft Maintenance Efficiency — MRO Management System
Standard Academic Compliance: IEEE Std 830-1998

SUBMITTED BY:
1. AYUSH SAHU (Roll No: 2400321540061)

Degree: Bachelor of Technology (B.Tech)
Academic Year: 2026 - 2027

UNDER THE GUIDANCE OF:
Mr. Abhi Gurjar
Project Guide
Department of Computer Science and Engineering – Data Science

Status: 3rd Year Project Specification
Date of Submission: 01-10-2026

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	15-Sep-2026	Initial Draft & Scope Definition	Project Team	Project Guide
1.0	01-Oct-2026	Final FlightFix SRS based on implemented project functionality	AYUSH SAHU	Project Guide

> Guidance Note: This Software Requirements Specification document conforms to the IEEE 830-1998 standard recommended practice. All sections are designed for 3rd-year university project evaluation, project review, and system design documentation.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, clear, and unambiguous definition of the functionality, external interfaces, performance targets, constraints, and implemented behavior of FlightFix: Aircraft Maintenance Efficiency — MRO Management System. This specification serves as a reference for the student developer, project guide, evaluators, and intended users of the academic project.

1.2 Scope of the System

The FlightFix application is designed to address the difficulty of managing aircraft maintenance activities, work orders, spare-parts inventory, technician information, and aircraft status through separate or manual records. It provides a centralized Django-based web platform for maintaining and monitoring these operational records.

- **In-Scope Capabilities: User authentication, dashboard access, aircraft management, aircraft search and status filtering, maintenance work-order management, inventory management, technician management, CRUD operations, status monitoring, dashboard statistics, and responsive web access.**
- **Out-of-Scope Boundaries: Physical aircraft inspection or repair, direct aircraft-control operations, real-time flight-control systems, production ERP integration, external enterprise API integration, and physical hardware interfacing.**
- **Expected Benefits: Centralized maintenance information, reduced manual record keeping, easier tracking of aircraft and maintenance status, improved spare-parts visibility, organized technician information, and faster access to operational records through a single web interface.**

1.3 Definitions, Acronyms, and Abbreviations

Category	Term / Acronym	Definition / Standard Context

Standard	SRS	Software Requirements Specification (IEEE Std 830-1998 format).
Architecture	Django MVT	Django Model-View-Template architecture used to separate data, application logic, and presentation.
Database	SQLite	Relational database used for local academic development and testing.
Application	CRUD	Create, Read, Update and Delete operations performed on FlightFix records.
Network	HTTP/HTTPS	Web communication protocols used by the Django application; HTTPS is recommended for deployed environments.
Project Specific	MRO	Maintenance, Repair and Overhaul activities associated with aircraft maintenance.

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Software Engineering (10th Edition):** Ian Sommerville, Pearson Education.
- **Relevant references: Django official documentation, Python documentation, SQLite documentation, IEEE Std 830-1998 guidance, and academic literature related to aircraft Maintenance, Repair and Overhaul (MRO) systems.**

1.5 Document Overview

The remainder of this SRS is partitioned into structural sections: Section 2 outlines the FlightFix product perspective, user classes, operating environment, design constraints, assumptions and dependencies. Section 3 defines the implemented functional requirements. Section 4 specifies the external interfaces. Section 5 defines non-functional requirements and verification methods. Section 6 identifies the standard system-design and analysis models required for academic evaluation.

2. Overall Description

2.1 Product Perspective

FlightFix is a self-contained Django-based web application comprising a presentation layer using Django templates, an application/business-logic layer implemented in Python/Django views, and a persistent SQLite database. The system provides authenticated access to aircraft, maintenance, inventory and technician management functions. No external ERP or third-party API integration is implemented in the current version.

> [System Context Diagram Placeholder]
> Insert the FlightFix Level-0 Context Diagram showing Administrator/Authenticated User, FlightFix system, and the SQLite database. The diagram should show authentication, aircraft management, maintenance management, inventory management, technician management, and dashboard information flow.

2.2 User Classes and Characteristics

The target software accommodates distinct user personas with varying access hierarchies:

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
Administrator	Intermediate	Authenticated administrative access to aircraft, maintenance, inventory and technician records; can perform implemented CRUD operations.
Authenticated User	Basic to Intermediate	Login-protected access to dashboard and operational modules according to the implemented application workflow.
Guest / Unauthenticated Visitor	Basic	Can access the public homepage and login interface; protected operational pages require authentication.

2.3 Operating Environment

The application operates within the following software and hardware environments:

- **Client Requirements: Modern web browser such as Google Chrome, Mozilla Firefox, Microsoft Edge or Safari; responsive layouts support desktop and mobile-sized screens.**
- **Server Runtime Environment: Python 3.13.7 virtual environment with Django framework on the local development machine.**
- **Database Storage Tier: SQLite database used by the current local FlightFix implementation.**

2.4 Design and Implementation Constraints

- **Academic Budget Constraint: Localhost-based development using open-source Python and Django technologies.**
- **Security Compliance: Django authentication is used for login/logout and protected pages; passwords are handled through Django's authentication framework. Production deployment should use HTTPS and appropriate access controls.**
- **Modularity Standards: The project follows Django's modular Model-View-Template structure with separate models, views, templates and static assets.**

2.5 Assumptions and Dependencies

- **Assumptions: Authorized users provide valid aircraft, maintenance, inventory and technician information; the local development environment and configured SQLite database are available during use.**

- **Dependencies: Python, Django, the configured SQLite database, local project files, and a modern web browser. No external ERP/API dependency is required by the current implementation.**

3. System Features and Functional Requirements

Requirements are categorized by functional subsystem modules with unique requirement identifiers (FR-x.x):

3.1 Module 1: Authentication and Access Governance

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	User Authentication	Username and password are required; valid credentials authenticate the user.	High
FR-1.2	Login / Logout	Successful login redirects to the dashboard; logout ends the authenticated session and redirects to login.	High
FR-1.3	Protected Access	Dashboard and operational modules require an authenticated user.	High

3.2 Module 2: Core Processing & Business Operations

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	Aircraft Management	Registration number, model, status and maintenance dates are stored and updated through CRUD workflows.	High
FR-2.2	Maintenance Work-Order Management	Aircraft, maintenance type, description, scheduled date and status are stored and editable.	High
FR-2.3	Inventory Management	Part number, part name, category, quantity and minimum stock are maintained; quantity determines stock status.	High

3.3 Module 3: Reporting, Visualization & Administrative Control

Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	Technician Management	Name, employee ID, specialization, experience, current task and status are maintained through CRUD operations.	High
FR-3.2	Dashboard Statistics	Aggregates aircraft status, pending maintenance and operational information for authenticated users.	Medium

FR-3.3	Search, Filter & Status Monitoring	Aircraft search/filter and status indicators provide quick access to current aircraft, maintenance, inventory and technician information.	Medium

4. External Interface Requirements

4.1 User Interfaces (UI)

The interface follows a clean dashboard-oriented design with navigation, searchable tables, status indicators, forms, action controls and responsive layouts. The implemented application provides desktop and mobile-friendly navigation and responsive data presentation for the main operational modules.

> [User Interface Screenshots Placeholder]
> Insert FlightFix screenshots for: (1) Login, (2) Dashboard, (3) Aircraft Management, (4) Maintenance Management, (5) Inventory Management, and (6) Technician Management.

4.2 Hardware Interfaces

Specification of physical interaction points:

- **Standard Deployment: Keyboard, mouse, display and mobile touchscreen devices may be used to access the web application.**
- **Embedded / Aircraft Hardware: No direct aircraft-control, sensor, microcontroller, or embedded hardware interface is implemented.**

4.3 Software Interfaces

- **Database Connector: Django ORM provides application access to the SQLite database.**
- **External Utilities: No external ERP, cloud-storage, payment, or third-party API integration is implemented in the current version.**

4.4 Communication Interfaces

- **Web Traffic Protocol: HTTP is used for controlled localhost development. HTTPS should be used for any future deployed environment.**
- **Data Handling: Django forms and HTTP requests are used for application interactions; Django ORM maps application data to SQLite records.**

5. Non-Functional Requirements (NFRs)

Attribute	Target Metric / Standard	Verification & Testing Method
5.1 Performance	Normal local pages and database operations should respond promptly under academic usage; no production load target is claimed for	Manual browser inspection and automated functional tests.

	the localhost version.	
5.2 Security	Authenticated access is required for protected operational pages; Django authentication handles credentials. Production hardening is outside current scope.	Authentication tests and manual access-control verification.
5.3 Reliability	Core CRUD, status logic, authentication and page-access behavior should remain consistent after valid transactions.	Manual testing plus Django automated test suite: 26/26 tests passed.
5.4 Portability	Application should operate on a modern browser and responsive desktop/mobile viewport in the supported local Python/Django environment.	Responsive viewport inspection and manual browser testing.

6. System Design and Analysis Models

University evaluation rubrics require standard graphical analysis models illustrating the dynamic and static behavior of the application:

Diagram Type	Objective & Scope	Deliverable Format
Use Case Diagram	Maps Administrator, Authenticated User and Guest interactions with authentication, aircraft, maintenance, inventory, technician and dashboard functions.	UML standard diagram + use-case specification tables.
Data Flow Diagram (DFD)	Illustrates data movement between users, FlightFix application modules and the SQLite database; Level 0 and Level 1 decomposition.	Gane-Sarson / Yourdon & Coad notation.
Entity-Relationship (ERD)	Shows User, Aircraft, Maintenance, Inventory and Technician entities, their attributes, keys and relationships.	Crow's Foot ERD / relational schema.
Activity / Sequence Diagram	Shows chronological interaction between user, Django views, models and database for core workflows such as login or maintenance creation.	UML activity/sequence diagram covering normal and validation/error paths.

7. Academic Review & Sign-Off

This Software Requirements Specification document has been prepared for the academic review of the FlightFix project and reflects the functionality implemented in the current localhost version.

Project Guide	Internal Examiner	Head of Department (HOD)
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____